AI Image Generator

4. Workflows

Version 2.2
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

4. Workflows

4.1 First-time setup

1. Confirm the GPU and driver are visible: run nvidia-smi in a terminal. It must list the card and a driver version.
2. Open PowerShell in the project folder.
3. Create and activate a virtual environment: `python -m venv .venv` then `.\.venv\Scripts\Activate.ps1`
4. Install the CUDA build of PyTorch, then the remaining dependencies: `pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126` and `pip install -r requirements.txt`
5. Verify CUDA is active: `python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"`
6. Confirm the upscaler weights exist at models\RealESRGAN_x4plus.pth. If not, download them from the Real-ESRGAN v0.1.0 release page into that folder.
7. Start the app once. The first start downloads the 20 GB Z-Image Turbo model into the Hugging Face cache. Other models download the first time they are used.

4.2 Daily use: generating an image

1. Double-click run-app.bat, or run `.\.venv\Scripts\python.exe app.py` from the project folder.
2. Wait for "Z-Image Turbo ready" and "Upscaler ready" in the console (about a minute). The browser opens http://127.0.0.1:7860 automatically.
3. Type a prompt. As you type, the line under the box shows the interpreted settings and the model, for example "1344x768 | upscale to 3840x2194 | 40 steps | guidance 5 · model Juggernaut XL v9".
4. Optionally pick a model in the Model box (Auto is the default) and edit the negative prompt.
5. Press Generate or Enter. A progress bar shows the model being loaded if it changed, then the step count and, if applicable, the upscale stage.
6. The image appears in the gallery. The Upscale bar appears under it, and the details panel shows timing, settings, GPU telemetry and the exact prompt sent to the model.
7. Both files are already saved in outputs\. Clear resets the prompt and results.



Figure 4.1: "a lighthouse on a rocky cliff at golden hour, cinematic, 16:9, 4k, seed 42" rendered at 1344x768 and upscaled to 3840x2194.

4.3 Choosing a model

- Say nothing: Auto keeps Z-Image Turbo (loaded at start) or FLUX. When an SDXL-family model is loaded it switches to the specialist for a style word if that model is on disk (photorealistic to Juggernaut, anime to Animagine, oil painting or watercolor to Playground, digital art, comic or cyberpunk to DreamShaper).
- Name it in the prompt: `model juggernaut`, `model: flux`, `checkpoint=realvis`, `using dreamshaper`, `with playground`.
- Pick it in the Model box. A prompt directive always wins over the box.
- The Models accordion under the prompt lists every model, its default steps and guidance, whether it is on disk, and what it is best for.
- Steps and guidance follow the model. Quality words still scale them: `quick` halves the steps, `highly detailed` adds 35 %, `strict` and `creative` move guidance by 30 %.

4.4 Using LoRAs and your own checkpoints

- Copy an SDXL LoRA file to models\loras\, for example pixel-style.safetensors, and write `<lora:pixel-style:0.8>` anywhere in the prompt. Several tags can be stacked; the weight is optional and defaults to 1.
- Copy an SDXL checkpoint (a single .safetensors file, as distributed on Civitai) to models\checkpoints\ and restart the app. It appears in the Model box and can be named in the prompt by its file name. Files with turbo, lightning or hyper in the name get 6-step defaults.
- Two LoRAs are included: pixel-art-xl and papercut. Tick them in the LoRAs box or tag them in the prompt. SDXL LoRAs apply to SDXL-family models; a LoRA that cannot be found or does not fit the loaded model is reported in the notes under the details report rather than failing the generation.

4.5 Producing 4K and 8K output

From the prompt: include 4k, 8k, 2k or an exact size such as 3840x2160. The parser sets an upscale target and raises steps by a third for a cleaner base. Generation runs at the native bucket for the aspect ratio, then the upscaler runs one x4 pass for 4K or two passes for 8K, and an exact size is centre-cropped.

After generation: the Upscale bar under the gallery offers 2K, 4K and 8K. Click an image to upscale only that one, or leave nothing selected to upscale the whole batch. The gallery switches to the upscaled files and the history rows gain the upscale columns. The native PNG is always kept.

Recommended practice: iterate at native size with a fast model until the composition is right, then upscale the chosen image from the bar. That costs 3 to 9 seconds instead of a regeneration.



Figure 4.2: The same 300x225 region of the lighthouse cliff. Left: native pixels enlarged. Right: Real-ESRGAN output. The upscaler reconstructs rock texture rather than blurring.

4.6 Reviewing past generations

1. Open the History tab. The top line shows totals: number of images, GPU seconds, average and extreme timings, VRAM, hottest temperature and peak power.
2. Type part of a prompt in the search box and press Enter to filter, or press Refresh to reload. The table now has a model column.
3. Click a row. The image (upscaled if one exists) and its full details appear below the table.
4. For scripted access, run `db.py`, `db.py 12` or `db.py stats` with the virtual environment's Python.

4.7 Adding a model or a LoRA

1. Open Admin, then Add. Paste a source: a Hugging Face repo id such as RunDiffusion/Juggernaut-XL-v9, a file in a repo such as nerijs/pixel-art-xl/pixel-art-xl.safetensors, a Civitai model link, a direct https link to a .safetensors file, or a local path. Press Check.
2. The result line says what was found: pipeline class and family for a diffusers repo, file name and size for a file, type, base model and trigger words for a Civitai entry, and warnings (gated repo, token needed, several files). The form is pre-filled: Add as (model, checkpoint file or LoRA), key, name, family, trigger words, default steps, guidance and sampler.
3. Press Add. A diffusers repo is registered at once and downloaded in the background if Download now is ticked; a file is downloaded into models/checkpoints/ or models/loras/ and registered when it finishes. The Downloads tab shows bytes, speed and remaining time; the header shows the number of running downloads.
4. The new model appears in the Model box (and answers to "model <key>" in the prompt) and the new LoRA in the LoRAs box without a restart. Small files can also be uploaded from the browser at the bottom of the Add tab.
5. To remove something: Models tab, select it, Remove from list (a built-in is reset to its defaults instead) or tick the confirmation and Delete files from disk; LoRAs tab, select it, tick the confirmation and Delete LoRA file.

4.8 Managing models and LoRAs

1. Models tab: click a row. Edit name, description, default steps and guidance, sampler, weight variant, aliases, auto-pick styles, negative-prompt support and Enabled, then Save changes. Load now loads it immediately

(with progress); Download in background fetches it without loading; Set as start-up model changes what loads when the app starts; Unload current model frees the VRAM.

2 LoRAs tab: click a row. Set the family it was trained for (SDXL, Z-Image, Qwen-Image or FLUX), its trigger words, default weight, description, source and tags, then Save. Try it in the Studio fills a test prompt with the trigger words, the LoRA tag and a matching model and switches to the Studio tab.

3 Settings tab: start-up model, the weight used for LoRAs ticked in the LoRAs box when they have no weight of their own, the default negative prompt, the Hugging Face token (required for FLUX and gated repos; Check login verifies it), the Civitai token, clean-up of generations older than N days, and the admin password lock.

4.9 Watching the machine: Overview, System and Logs

1 Admin, Overview: system tiles, activity tiles (images today and this week, GPU minutes in 24 h, loaded and start-up model, models on disk, LoRAs, downloads, outputs size, Hugging Face login) and the per-model statistics table.

2 Admin, System: the nine tiles and six detail sections of section 9.4. Press Refresh for a new sample, or tick Auto-refresh every 5 s while a generation runs to watch VRAM, temperature and power move; statistics keep refreshing during generation.

3 Tick "Check internet reachability" to add a TCP connect test to Hugging Face, GitHub and PyPI; it is off by default because an offline machine would wait 2 s per host.

4 Admin, Logs: the last lines the app printed (model loads, generations, LoRA notes, downloads, errors) with a text filter, an errors-only level and auto-refresh.

4.10 Batch generation from the command line

```
.\.venv\Scripts\python.exe generate.py --list-models
.\.venv\Scripts\python.exe generate.py "a red sports car" --model juggernaut --seed 42 --count 4
.\.venv\Scripts\python.exe generate.py "cat astronaut" --model dreamshaper --lora pixel-style:0.7
.\.venv\Scripts\python.exe generate.py --help
```

The script loads the model per run, so it is best for batches. It writes to outputs\ but does not record to the database or upscale.

4.9 Upscaling an existing image

```
.\.venv\Scripts\python.exe upscaler.py outputs\some_image.png 3840
```

The result is written next to the input with an _up3840 suffix. For images made in the app, prefer the Upscale bar so the history row is updated too.

4.10 Enabling FLUX.1

1 Accept the licence at https://huggingface.co/black-forest-labs/FLUX.1-schnell while logged in to Hugging Face.

2 Run `.\.venv\Scripts\huggingface-cli login` and paste a read token.

3 Say `model flux` in a prompt. The first use downloads about 33 GB; on a 24 GB card the transformer is quantised to 4-bit and the T5 encoder to 8-bit at load time, which takes a few minutes.

4 FLUX ignores the negative prompt, guidance words and LoRAs. Four steps are enough.

4.11 Running in Docker

1 Install the prerequisites from section 11 (on Windows: WSL 2 and Docker Desktop).

2 In the project folder run `docker compose up --build`. The first build installs PyTorch and the dependencies (several GB).

3 Open http://localhost:7860. Images land in outputs\, the database is generations.db, and downloaded models persist in the hf-cache volume.

4 Stop with Ctrl+C or `docker compose down`. The volume survives; `docker compose down -v` deletes the cached models.

4.12 Stopping, restarting and backup

- Close the console window, or press Ctrl+C in it, to stop the server. Loaded models are released.
- Restart with run-app.bat. Startup is about 30 to 45 seconds because the model is reloaded from disk.
- If port 7860 is reported busy, a previous instance is still running; end it from Task Manager (python.exe) or set GRADIO_SERVER_PORT.
- Back up the outputs\ folder, generations.db and models\ together. Built-in models can be re-downloaded and the virtual environment rebuilt from requirements.txt.

4.14 Generating a video clip

1 Open the Video tab. Describe the scene, the motion and the camera in one or two sentences; the chips add camera and motion hints (tracking shot, drone shot, slow motion, golden hour ...).

2 Pick the model (LTX-Video 2B by default; Wan 2.1 for smoother but slower motion), a size, a length in seconds, and optionally steps, guidance and a seed. The model box resets steps, guidance and the negative prompt to the model's defaults.

3 Optional: drop a start image into the Start image box (LTX only). It is centre-cropped to the chosen size and animated (image-to-video).

4 Press Generate video. The first use unloads the image model, loads the video model (about 2 minutes the first time, about a minute afterwards) and shows step progress; the clip plays in the result box and its details appear below.

5 Optional: Upscale video (Full HD, 2K or 4K) runs every frame through Real-ESRGAN and re-encodes the clip; Add soundtrack lays a track from the Music tab under it. Both produce new files next to the original.

6 Clips are in outputs/ (<timestamp>_seed<seed>.mp4, _up<px>.mp4, _music.mp4) and in History under kind = video.

4.15 Generating a music track

1 Open the Music tab. Describe genre, mood, instruments and tempo; the chips add common genres and moods.

2 Pick the model (MusicGen stereo medium by default), the length (3 to 30 s), guidance, temperature and optionally a seed.

3 Press Generate music. MusicGen loads in about 30 s (it can stay loaded next to an SDXL model) and writes the track with a progress bar; the player appears with the details below.

4 Tracks are 16-bit WAV files in outputs/ and appear in History under kind = music with a waveform thumbnail. The Video tab lists them in its Soundtrack box.

4.13 Development workflow

1 Edit a module. Self-tests: `python prompt_parser.py` prints how sample prompts are interpreted, `python models.py` lists the registry and what is on disk, `python registry.py` prints the registry file, `python downloads.py <source>` inspects a source, `python sysinfo.py` dumps the Admin data as JSON, `python db.py stats` exercises the schema.

2. Compile-check with `python -m py_compile app.py admin_ui.py media_ui.py media.py video_io.py models.py registry.py downloads.py prompt_parser.py sysinfo.py`. `python media.py` lists the video and music models and whether they are on disk.
3. Restart the app. For automated checks, gradio_client can call /generate, /upscale, /refresh_history, /overview_refresh, /system_refresh, /check_source, /add_source, /save_lora and /logs_text, which is how the increments in this project were verified.
4. Rebuild this documentation with `python docs-src\build_pdf.py`.
5. Commit and push: the repository is https://github.com/Fzhacking/ai-image-gen.